

PDF 요약 앱 — 세션 회고: 에이전트별 실적 및 멀티에이전트 활용 평가

- 작성일: 2026-07-28
- 범위: 브레인스토밍 → 계획 수립 → 서브에이전트 기반 구현(SDD) → 배포까지 전체 세션

서브에이전트별 작업 실적

product-manager — Task 1 (PRD 작성)

- 산출물: docs/PRD.md (8개 섹션, 성공 지표, 기능 요구사항, 에러 케이스 표 등)
- 결과: DONE_WITH_CONCERNS → 리뷰에서 Important 1건(잘못된 내부 절 참조) 발견 → 1회 수정 후 승인
- 비고: 툴셋에 Bash가 없어 git 커밋은 코디네이터가 대신 수행. 콘텐츠 품질 자체는 높았음(설계 문서 결정을 임의로 뒤집지 않고 충실히 구체화, TBD 없음).

backend-developer — Task 2, 3, 4

태스크	결과	비고
Task 2 (Next.js 스캐폴딩)	DONE_WITH_CONCERNS → 승인	.gitignore 에서 .worktrees/ 항목이 브리프 교체로 유실되는 걸 스스로 발견해 보고. npm audit 경고는 정당하게 보류
Task 3 (PDF 텍스트 추출)	DONE_WITH_CONCERNS(2회) → 승인	textutil 이 이 macOS에서 PDF 출력 미지원임을 발견(브리프 오류였음). 처음엔 reportlab으로 우회하려다 한글이 깨졌고, 코디네이터 조사 후 cupsfilter + 영어 텍스트로 재검증
Task 4 (API 라우트)	DONE(클린)	실제 curl로 정상/에러 경로 모두 검증, 이슈 없음
최종 리뷰 수정 웨이브	DONE	Critical 1건(4MB 한도) + Important 3건(폼데이터 가드, 클라이언트 에러 처리, points 검증)을 한 번에 수정하고 실제 재검증(경계값 curl 등)

ai-integration-specialist — Task 5 (OpenRouter 연동)

- 결과: DONE(클린, 리뷰 이슈 없음)
- 하이라이트: 실제 라이브 API 호출로 검증(4번째 폴백 모델까지 실제로 타는 것을 확인), FREE_MODELS 목록이 여전히 유효한지 자체적으로 재확인(브리프의 "확인 후 진행" 지시를 문

자 그대로 따름), 응답 지연이 20초 이상 걸릴 수 있다는 UX 리스크를 다음 태스크(프론트엔드)에 미리 공유

frontend-developer — Task 6, 7

태스크	결과	비고
Task 6 (업로드 UI)	DONE	Playwright를 직접 설치해 실제 브라우저로 4개 상태, 키보드 접근성 전부 스크린샷과 함께 검증. 덤으로 모델이 간헐적으로 영어로 응답하는 현상을 관찰해 보고
Task 7 (결과 컴포넌트)	DONE	코디네이터가 부정확하게 지시한 "catch 블록" 위치를 스스로 검증해 실제로 맞는 위치(!res.ok 분기)에 수정 적용 — 지시를 맹목적으로 따르지 않고 근거로 반박한 사례

qa-engineer — Task 8 (엡지케이스 QA)

- **결과:** DONE(8/8 시나리오 최초 통과, 버그 0건)
- **비고:** 최종 리뷰에서 QA 리포트 자체의 증거 품질(일부 시나리오가 "동일 메시지"로만 기록되어 실제 응답 원문이 부족)에 대한 Minor 지적을 받음 — 결과는 정확했으나 기록 방식은 개선 여지가 있었음

리뷰 에이전트(general-purpose, 태스크별 8회 + 최종 1회)

- 태스크별 리뷰 8회 중 7회는 Approved, 1회(Task 1)는 1회 수정 후 승인
- **최종 전체 브랜치 리뷰**(opus)가 가장 큰 가치를 만들어냄: 업로드 제한 10MB가 Vercel 함수 요청 본문 한도(~4.5MB)를 초과해 실제 배포 시 절대 동작할 수 없는 Critical 결함을 발견. 이건 어떤 개별 태스크의 범위에도 속하지 않아 태스크별 리뷰로는 절대 잡을 수 없었던 종류의 문제였음. 그 외 Important 3건(무가드 formData, 클라이언트 에러 순서 버그로 타임아웃 메시지 미구현, points 검증 누락+필터 순서 버그)도 전부 이 단계에서 처음 발견됨

배포 후 발견된 추가 문제 (에이전트 위임 없이 코디네이터가 직접 처리)

병합, 정리까지 끝낸 뒤 실제 Vercel 배포, 검증 요청을 받고 나서야 드러난 문제들:

1. 한글(CJK) PDF 텍스트 추출이 실제 프로덕션에서도 실패(502) — 원인 조사 중 두 번 잘못된 진단(처음엔 "로컬 폴더 경로의 한글/특수문자", 다음엔 "로컬 Node 버전의 fetch file:// 미지원")을 내렸다가, 실제로는 pdfjs-dist 가 정식 의존성으로 추가된 적이 없다는 게 진짜 원인을 확인. 이후 세 가지 방식(정식 의존성 추가만 / public 정적 파일 + HTTP self-fetch / outputFileTracingIncludes + 절대경로 file:// 직접 지정)을 순차 시도해 크래시(502)는 해결

했으나, 텍스트가 완전히 정상 추출되는지는 OpenRouter 무료 티어 일일 한도 소진으로 최종 확인 못함(다음날 재확인 필요)

2. 세션 내내 반복적인 테스트로 OpenRouter 무료 계정의 일일 요청 한도(50회/일)를 소진 — 앱 코드 문제가 아니라 계정 자체의 제약

멀티에이전트(서브에이전트 기반 SDD) 활용 방식 효용성 평가

잘 작동한 점 - 역할별 에이전트(PM/백엔드/AI연동/프론트엔드/QA)가 각자 영역에 맞는 산출물 품질을 보였다 — PM은 성공 지표가 명확한 PRD를, QA는 증거 기반 리포트를, 프론트엔드는 실제 브라우저 검증을 자발적으로 수행 - 에이전트들이 코디네이터의 지시를 맹목적으로 따르지 않고 근거를 들어 정정한 사례가 두 번 있었음(Task 5의 모델 목록 재검증, Task 7의 위치 정정) — 단순 실행이 아니라 검증하는 태도가 실제로 관찰됨 - "태스크별 리뷰 + 전체 브랜치 최종 리뷰"의 2단계 구조가 실질적 가치를 냄: 최종 리뷰가 아니었다면 10MB 업로드 한도(Vercel 4.5MB 초과) 버그가 그대로 프로덕션에 배포됐을 것 - 격리된 git worktree에서 작업해 `main` 브랜치가 중간 상태로 오염되지 않았고, 정리도 깔끔하게 됨

한계로 드러난 점 - 아무리 철저한 로컬 검증(빌드, 타입체크, 8개 QA 시나리오, 전체 리뷰)도 **실제 배포 환경 자체를 대체하지 못했다**. 한글 CJK 추출 실패와 Vercel 함수 본문 크기 한도 문제 둘 다, 실제 배포 후에야 (하나는 최종 리뷰가, 하나는 실제 curl 테스트가) 발견됨 — "배포해서 확인"이 형식적 마무리가 아니라 검증 파이프라인의 필수 마지막 단계여야 한다는 교훈 - 로컬 개발 환경 자체의 특이성(매우 최신 Node 버전, 한글, 공백이 섞인 폴더 경로, macOS `textutil`의 PDF 미지원)이 반복적으로 잘못된 진단을 유발했다 — 이런 환경 특이 이슈는 에이전트에게 통째로 위임하기보다 사람(또는 코디네이터)이 직접 격리, 재현하며 판단해야 하는 경우가 많았음 - 작은 규모 앱(소스 파일 10여 개) 치고는 에이전트 디스패치 횟수가 상당히 많았다(8개 태스크 × 구현+리뷰(+수정 라운드) + 최종 리뷰 + 수정 웨이브 + 재검증 ≈ 20회 이상). 학습, 신뢰성 확보 목적에는 그 비용이 정당화되지만, 사소한 변경에는 과할 수 있는 구조 - "배포 후 재확인" 항목(한글 처리)이 정책으로는 명시됐지만, 병합 완료 시점에 "끝났다"고 느끼기 쉬워 실제로 배포해서 재확인하기 전까지는 잠재 결함이 조용히 남아있었음 — 유예 항목은 반드시 추적 가능한 후속 액션으로 남겨야 함

총평: 이번 세션에서 멀티에이전트 구조는 실제로 배포 가능한 수준의 작동하는 앱을 만들어냈고, 특히 "리뷰 단계"가 실질적인 결함(Critical 1건 포함)을 잡아낸 것은 명확한 성과다. 다만 모든 검증이 로컬/시뮬레이션 기반이었던 한계로, 진짜 프로덕션 환경에서만 드러나는 문제(배포 플랫폼 제약, 서버리스 번들링 특성)까지는 잡아내지 못했다 — 이는 에이전트 활용 방식의 결함이라기보다, "실제 배포 검증"을 파이프라인의 정식 마지막 단계로 넣지 않은 설계상의 공백에 가깝다.